

Supplementary material for the work:

“Thermodynamic and structural properties of the thin atmosphere of a white dwarf”

Riccardo Fantoni*

Università di Trieste, Dipartimento di Fisica, strada Costiera 11, 34151 Grignano (Trieste), Italy

(Dated: September 24, 2026)

Supplementary material to the article “Thermodynamic and structural properties of the thin atmosphere of a white dwarf”.

Keywords: White dwarf;Jellium;Path Integral;Monte Carlo;General Relativity;Thin surface layer;Thermodynamics;Structure

I. INTRODUCTION

We here report the FORTRAN code listing of the code used in the simulations developed in the publication “Thermodynamic and structural properties of the thin atmosphere of a white dwarf”. This is shown in Appendix A.

ACKNOWLEDGMENTS

I would like to thank prof. Saverio Moroni for his support in the development of the Brownian bridge and the consequent particles permutation sampling.

Appendix A: The code

This is the code used for the RPIMC computer experiment. We list here the main FORTRAN code `rpimc-wd-rs.f` with the included `mc-wd.par` parameters file. The input simulation configuration file `data-gr-rs.in` that is read at the beginning of the run. The subroutine `ifdm.f` to build the free particle density matrix used to enforce the restriction in the RPIMC algorithm. And the `Makefile`.

```
*****
*** Makefile *****
*****

OBJ = rpimc-wd-rs.o ifdm.o

.f.o:
gfortran -c $<

rpimc-wd-rs.o: mc-wd.par rpimc-wd-rs.f
gfortran -c -freal-8-real-16 rpimc-wd-rs.f

ifdm.o: ifdm.f
gfortran -c ifdm.f

rpimc: $(OBJ)
gfortran -o rpimc $(OBJ)

clean:
- rm -f *.o *~

*****
*** rpimc-wd-rs.f *****
*****

PROGRAM PIMC
IMPLICIT NONE
```

```
C *****
C ** MONTE CARLO SIMULATION PROGRAM FOR RESTRICTED PATH INTEGRAL **
C **
C ** in the atmosphere of tickness DELTA of a white dwarf of **
C ** radius RADIUS and mass MASS **
C **
C ** space dimensions: DIM=3 **
C ** number of particles: NP **
C ** number of timesteps: FTNO **
C ** units: hbar=k_B=1 **
C ** BETA=1/TEMP=FTNO*LS (LS imaginary time spacing) **
C **
C ** OBSERVABLES: **
C ** kinetic energy KE **
C ** potential energy V **
C ** forces virial W **
C ** superfluid fraction SF **
C ** radial distribution function on 'rdf.dat' **
C *****
C INCLUDE 'mc-wd.par' ! parameters

INTEGER*4 SEED
INTEGER*8 IDIM, STEP, I, J, K
INTEGER*8 C1, CP, IP, KP, NIP, NKP, LL, OUT1, OUT2, PIP
INTEGER*8 INIT, NSTEP, NS, IPRINT, ISAVE, IRATIO, IEQUI
INTEGER*8 MBMM, MCM, PREV(MNP)
INTEGER*8 RS, CYC(MNP), NSWAP

REAL*8 TEMP, BETA, DENSLJ, v1,v2,w1,w2
REAL*8 DRMAX(MDIM), ALPHA(MDIM), CDIM
REAL*8 RANF, DUMMY, SR9, SR3
REAL*8 VLRC, VLRC6, VLRC12, WLRC, WLRC6, WLRC12
REAL*8 ACM, ACATMA, ACATMAS, ACATMAB
REAL*8 V, VNEW, VOLD, VEND, DELTV, VN, VS
REAL*8 W, WNEW, WOLD, WEND, DELTW, WN, WS
REAL*8 KE, KENEW, KEOLD, DELTKE, KEEND
REAL*8 AVV, ACV, ACVSQ, FLV
REAL*8 AVW, ACW, ACWSQ, FLW
REAL*8 AVKE, ACKE, ACKESQ, FLKE
REAL*8 ACT, ACTNEW, ACTOLD, DELTACT, DELTACTB
REAL*8 RXIOLD(MDIM), RXINEW(MDIM), RO(MDIM,0:N,MNP)
REAL*8 RXP(MDIM,0:N), RXPP(MDIM,0:N)
REAL*8 STDR, STDA, PS, KKK, VVV, WWW
REAL*8 RATIO, RATIOS, RATIOB
REAL*8 WIND, ACWIND, SFI, SF, IC
REAL*8 GA, IPDMS, EPSILON

INTEGER*8 BIN, MAXBIN, MAXBIN2, HIST(1000000), ICALOG
REAL*8 CONST, RLOWER, RUPPER
REAL*8 TSTEP, NIDEAL, GR

CHARACTER CNFILE*30, POTK*10

LOGICAL OVRLAP, IFB, IFDISP, IFBRIDGE

C PI = ACOS(-1.d0)

OPEN(UNIT = 16, FILE = 'back.dat', STATUS = 'UNKNOWN')
OPEN(UNIT = 15, FILE = 'rdf.dat', STATUS = 'UNKNOWN')
EPSILON = 0.

C *****
C ** READ INPUT DATA **
C *****

WRITE(*,(' ***** PROGRAM PIMC ***** '))
WRITE(*,(' PLANE WAVES '))
WRITE(*,(' PATH INTEGRAL MONTE CARLO PROGRAM '))
OPEN(UNIT=10, FILE='data-gr-rs.in', STATUS='UNKNOWN')
READ(10,*) I
READ(10,*) DIM
READ(10,*) I
READ(10,*) SEED
READ(10,*) I
READ(10,*) IFB
READ(10,*) I
READ(10,*) NP
READ(10,*) I
READ(10,*) POTK
READ(10,*) I
READ(10,*) FTNO
READ(10,*) I
READ(10,*) FTM
READ(10,*) I
READ(10,*) NSTEP
READ(10,*) I
READ(10,*) IPRINT
```

* riccardo.fantoni@scuola.istruzione.it

```

READ (10,*) I
READ (10,*) ISAVE
READ (10,*) I
READ (10,*) IEQUI
READ (10,*) I
READ (10,*) IRATIO
READ (10,*) I
READ (10,*) ICALCG
READ (10,*) I
READ (10,*(A')) CNFILE
READ (10,*) I
READ (10,*) INIT
READ (10,*) I
READ (10,*) RADIUS
READ (10,*) I
READ (10,*) MASS
READ (10,*) I
READ (10,*) DELTA
READ (10,*) I
READ (10,*) ANG
READ (10,*) I
READ (10,*) TEMP
READ (10,*) I
READ (10,*) (ALPHA(J),J=1,DIM)
READ (10,*) I
READ (10,*) MBMM
READ (10,*) I
READ (10,*) RCUT
READ (10,*) I
READ (10,*) SIG
READ (10,*) I
READ (10,*) EPSR
READ (10,*) I
READ (10,*) EPSA
READ (10,*) I
READ (10,*) EPS
READ (10,*) I
READ (10,*) IFDISP
READ (10,*) I
READ (10,*) IFBRIDGE
CLOSE (UNIT=10)
WRITE(*,(' IN DIMENSION (1,2,3,...)'I12/')) DIM
WRITE(*,(' ***** '))
GOTO 1111 ! comment if input from keyboard

WRITE(*,(' ***** PROGRAM PIMC ***** '))
WRITE(*,(' PLANE WAVES '))
WRITE(*,(' PATH INTEGRAL MONTE CARLO PROGRAM '))

WRITE(*,(' ENTER THE SPATIAL DIMENSIONS '))
READ (*,*) DIM
WRITE(*,(' ENTER SEED FOR RANDOM SEQUENCE '))
READ (*,*) SEED
WRITE(*,(' IF BOSE (T/F) '))
READ (*,*) IFB
WRITE(*,(' ENTER THE NUMBER OF PARTICLES < MNP '))
READ (*,*) NP
WRITE(*,(' ENTER TYPE OF PAIR POTENTIAL (Many Body) '))
READ (*,*(A')) POTK
WRITE(*,(' ENTER THE NUMBER OF DISCRETIZATIONS < N '))
READ (*,*) FTNO
WRITE(*,(' ENTER THE BARE MASS '))
READ (*,*) FTM
WRITE(*,(' ENTER NUMBER OF CYCLES '))
READ (*,*) NSTEP
WRITE(*,(' ENTER NUMBER OF STEPS BETWEEN OUTPUT LINES '))
READ (*,*) IPRINT
WRITE(*,(' ENTER NUMBER OF STEPS BETWEEN DATA SAVES '))
READ (*,*) ISAVE
WRITE(*,(' ENTER NUMBER OF STEPS FOR EQUILIBRATION '))
READ (*,*) IEQUI
WRITE(*,(' ENTER INTERVAL FOR UPDATE OF MAX. DISPL. '))
READ (*,*) IRATIO
WRITE(*,(' ENTER INTERVAL FOR RDF CALCULATION '))
READ (*,*) ICALCG
WRITE(*,(' ENTER THE CONFIGURATION FILE NAME '))
READ (*,*(A')) CNFILE
WRITE(*,(' ENTER 0 INIT Z, 1 INIT, 2 REST C, 2 REST F '))
READ (*,*) INIT
WRITE(*,(' ENTER THE WD RADIUS '))
READ (*,*) RADIUS
WRITE(*,(' ENTER THE WD MASS '))
READ (*,*) MASS
WRITE(*,(' ENTER THE WD ATMOSPHERE THICKNESS '))
READ (*,*) DELTA
WRITE(*,(' ENTER THE WD ANGULAR EXTENT OF THE PATCH '))
READ (*,*) ANG
WRITE(*,(' ENTER THE TEMPERATURE '))
READ (*,*) TEMP
WRITE(*,(' ENTER MAX. DISPLACEMENT DRMAX/SIGMA '))
READ (*,*) (ALPHA(J),J=1,DIM)
WRITE(*,(' ENTER MAXIMUM NUMBER OF BRIDGE TIMESLICES '))
READ (*,*) MBMM
WRITE(*,(' ENTER THE POTENTIAL CUTOFF DISTANCE ( 2.5) '))
READ (*,*) RCUT
WRITE(*,(' ENTER SIG '))
READ (*,*) SIG
WRITE(*,(' ENTER EPSR '))
READ (*,*) EPSR
WRITE(*,(' ENTER EPSA '))
READ (*,*) EPSA
WRITE(*,(' ENTER EPS '))
READ (*,*) EPS
WRITE(*,(' IF DISPLACEMENT MOVE (T/F) '))
READ (*,*) IFDISP
WRITE(*,(' IF BRIDGE MOVE (T/F) '))
READ (*,*) IFBRIDGE
WRITE(*,(' IN DIMENSION (1,2,3,...)'I12/')) DIM
WRITE(*,(' ***** '))

1111 CONTINUE

IF (MBMM.GT. FTNO) THEN
  WRITE(*,*) "number of timeslices in bridge > ",FTNO
  STOP
ENDIF

IF (DIM.NE. 3) THEN
  WRITE(*,*) "DIM not equal to 3"
  STOP
ENDIF

C ** INITIALIZE RANDOM NUMBER GENERATOR **

CALL SRAND ( SEED )
IF (SEED.EQ.1) THEN
  CALL SRAND ( 0 )
ENDIF

C ** DENSITY **

DENS = NP/(4.d0*PI*RADIUS**2.*DELTA)/(2.*SIN(ANG/2.)*ANG)

C ** INVERSE TEMPERATURE BETA **

BETA = (1.d0/TEMP)

C ** IMAGINARY TIMESTEP **

LS = BETA/FTNO ! time step
STDR = (LS/FTM)**0.5 ! standard deviation of free-particle
STDA = STDR/RADIUS

C ** MOMENT OF INERTIA OF A UNITARY SPHERICAL SHELL **

IC = 2*(NP*FTM)/3

C ** WRITE INPUT DATA **

WRITE(*,(' SEED ',I10 )) SEED
WRITE(*,(' POTENTIAL ',A )) POTK
WRITE(*,(' DIMENSIONS ',I10 )) DIM
WRITE(*,(' NUMBER OF PARTICLES ',I10 )) NP
WRITE(*,(' NUMBER OF CYCLES ',I10 )) NSTEP
WRITE(*,(' NUMBER OF EQUIL. STEPS ',I10 )) IEQUI
WRITE(*,(' OUTPUT FREQUENCY ',I10 )) IPRINT
WRITE(*,(' SAVE FREQUENCY ',I10 )) ISAVE
WRITE(*,(' RATIO UPDATE FREQUENCY ',I10 )) IRATIO
WRITE(*,(' CONFIGURATION FILE NAME ',A )) CNFILE
WRITE(*,(' TEMPERATURE ',E10.4 )) TEMP
WRITE(*,(' DENSITY ',E10.4 )) DENS
WRITE(*,(' PARTICLE MASS ',E10.4 )) FTM
WRITE(*,(' # TIME SLICES ',I10 )) FTNO
WRITE(*,(' TIME STEP ',E10.4 )) LS

C ** CONVERT INPUT DATA TO PROGRAM UNITS **

SIGMA = DELTA
SIGM(1) = ANG
SIGM(2) = ANG
DRMAX(1) = ALPHA(1) * SIGMA
DRMAX(2) = ALPHA(2) * SIGM(1)
DRMAX(3) = ALPHA(3) * SIGM(2)

IF (POTK.EQ. 'LJ') THEN
  DENSLJ = DENS * SIG ** DBLE( DIM )
  RCUT = SIGMA/2.d0/SIG
ENDIF

MAXBIN = 1000
DELR = DELTA/MAXBIN
MAXBIN2 = INT ( MAXBIN/2.0 )
DO I = 1, MAXBIN
  HIST(I) = 0
ENDDO

IF (DIM.EQ. 3) CONST = 4.d0 * PI * DENS / 3.d0
IF (DIM.EQ. 2) CONST = PI * DENS
IF (DIM.EQ. 1) CONST = 2.d0 * DENS

C ** INITIALIZE CONFIGURATION **

DO I = 1, NP
  NEXT(I) = I
  LEXT(I) = I
  DO J = 1, FTNO
    MEXT(J,I) = I
  ENDDO
ENDDO

C ** ZERO ACCUMULATORS **

ACV = 0.0
ACVSQ = 0.0
FLV = 0.0
ACW = 0.0
ACWSQ = 0.0
FLW = 0.0
ACKE = 0.0
ACKESQ = 0.0
FLKE = 0.0
ACWIND = 0.0
ACM = 0.0
ACATMA = 0.0
ACATMAB = 0.0
ACATMAS = 0.0

IF ( INIT.EQ. 0 ) THEN
  PRINT*, "PATHS INITIALIZATION ....."
  RX(1,:)=RADIUS-DELTA
  RX(2,:)=0. ! all paths zeroed
  RX(3,:)=0. ! all paths zeroed

```

```

      NS = NSTEP
      OPEN (UNIT = 9, STATUS = 'UNKNOWN')
      ELSEIF (INIT .EQ. 1) THEN
        PRINT*, "PATHS INITIALIZATION ....."
C      ** READ INITIAL CONFIGURATION **
        CALL INITCWD ( CNFILE ) ! random configuration on radius
        CALL READCN ( CNFILE )
        NS = NSTEP
        OPEN (UNIT = 9, STATUS = 'UNKNOWN')
        ELSEIF (INIT .EQ. 2) THEN
          PRINT*, "RESTART FROM CONFIG ....."
C      ** READ INITIAL CONFIGURATION **
        CALL READCN ( CNFILE )
        NS = NSTEP
        OPEN (UNIT = 9, STATUS = 'OLD', ACCESS = 'APPEND')
        ELSEIF (INIT .EQ. 3) THEN
          PRINT*, "RESTART FULL ....."
          CALL READCN ( CNFILE )
          ACATMA = 0.0
          ACATMA = 0.0
          ACATMA = 0.0
          READ(16,*) (HIST(I), I=1, MAXBIN),
&          ACV, ACWSQ, ACW, ACWSQ, ACKE, ACKESQ, ACWIND, ACM, NS
          NS=NS+NSTEP
          OPEN (UNIT = 9, STATUS = 'OLD', ACCESS = 'APPEND')
        ELSE
          WRITE(*,*) 'INIT different from 0,1,2,3'
          STOP
        ENDIF

C      ** CALCULATE LONG RANGE CORRECTIONS **
C      ** SPECIFIC TO THE LENNARD JONES FLUID **
      IF (DIM .EQ. 1) CDIM = 1
      IF (DIM .EQ. 2) CDIM = PI
      IF (DIM .EQ. 3) CDIM = 2*PI

      SR3 = - RCUT ** (- 6. + DIM)/(-6. + DIM)
      SR9 = - RCUT ** (- 12. + DIM)/(-12. + DIM)

      VLRC12 = 4 * EPS * CDIM * DENS LJ * NP * SR9
      VLRC6 = - 4 * EPS * CDIM * DENS LJ * NP * SR3
      VLRC = VLRC12 + VLRC6
      WLRC12 = 4.0 * VLRC12
      WLRC6 = 2.0 * VLRC6
      WLRC = WLRC12 + WLRC6

C      ** WRITE OUT SOME USEFUL INFORMATION **
      WRITE(*,*) 'SIGMA', 'E10.4') SIGMA
      WRITE(*,*) 'MAXIMUM DISPLACEMENT', 'E10.4') DRMAX(1)

C      ** CALCULATE INITIAL ENERGY AND CHECK FOR OVERLAPS **
      CALL SUMUP (POTK, OVRLAP, KE, V, W)

      IF (POTK .EQ. 'LJ') THEN
        VS = ( V + VLRC )
        WS = ( W + WLRC )
      ELSE
        VS = V - W
        WS = W
      ENDIF

      WRITE(*,*) 'INITIAL V', 'E10.4') V /FTNO
      WRITE(*,*) 'INITIAL V - W', 'E10.4') VS/FTNO
      WRITE(*,*) 'INITIAL W', 'E10.4') WS/FTNO
      WRITE(*,*) 'INITIAL KE', 'E10.4') KE/FTNO

      WRITE(*,*) 'START OF MARKOV CHAIN', '///')
      WRITE(*,*) 'NMOVE RATIO ACTION', '///')

C      *****
C      ** LOOPS OVER ALL CYCLES AND ALL TIME SLICES **
C      *****

      DO 100 STEP = NS-NSTEP+1, NS
        CP = INT(FTNO*РАНF(DUMMY))+1 ! select a random timeslice
        RS = INT(FTNO*РАНF(DUMMY))+1 ! select a random referenceslice
        RS = 1
        MEM = INT((MEMM-1)*РАНF(DUMMY))+2 ! # timeslices in bridge
        IP = INT(NP*РАНF(DUMMY))+1 ! select a particle
        KP = INT(NP*РАНF(DUMMY))+1 ! select another particle
        IF (.NOT. IFB) KP = IP ! for boltzmann statistics
        IF (IFDISP) GOTO 77 ! uncomment if only displacement move

C      *****
C      ** BRIDGE&SWAP MOVE (BOSE STATISTICS) **
C      *****

C      direct mapping from white dwarf atmosphere rx to plane rp
      call equiv(r0,rx)
      call mappingd()
      call equiv(rpn,rp)

      mcm = cp+mcm-floor((cp+mcm-.1)/ftn0)*ftn0

      nip=next(ip)
      nkp=next(kp)

      call sumupot ( potk, overlap, vold, wold )

      keold=0.d0
      do k=1,np
        if(k.eq.ip.or.k.eq.kp.or.k.eq.nip.or.k.eq.nkp)then
          do i=1,ftn0
            call kkenergy ( rx(:,i,k), k, i,
:              kkk )
              keold=keold+kkk
            enddo
          endif
        enddo

      c      bridge move
      if(cp+mcm.le.ftn0)then
        call bridge(rp(:,cp,ip),rp(:,mcm,kp),stdr,stda,
:          rxp,out1)
        if (ip.ne.kp) then
          call bridge(rp(:,cp,kp),rp(:,mcm,ip),stdr,stda,
:            rxpp,out2)
        endif
      else
        call bridge(rp(:,cp,ip),rp(:,mcm,nkp),stdr,stda,
:          rxp,out1)
        if (ip.ne.kp) then
          call bridge(rp(:,cp,kp),rp(:,mcm,nip),stdr,stda,
:            rxpp,out2)
        endif
      endif

      if (out1.eq.3.or.out2.eq.3) then
        out1 = 0
        out2 = 0
        goto 1235
      endif

      c      execute the bridge move
      acatmab = acatmab + 1.d0
      call updaten(cp,rxp,ip,nkp)
      if (ip.ne.kp) then
        call updaten(cp,rxpp,kp,nip)
        call swapn(cp,ip,nip,kp,nkp)
        call switch(next(ip),next(kp))
        call switch(next(cp,ip),next(cp,kp))
        acatmas = acatmas + 1.d0
      endif

      call equiv (rx,rpn)
      call sumupot ( potk, overlap, vnew, wnew )
      deltv=vnew-vold
      deltaw=wnew-wold

      c      calculate the ratio of transition probabilities
      call gauss(cp,ga)

      c      inverse mapping
      call equiv(rp,rpn)
      call equiv(rpn,rx)
      call mappingi()

      kenew=0.d0
      do k=1,np
        if(k.eq.ip.or.k.eq.kp.or.k.eq.nip.or.k.eq.nkp)then
          do i=1,ftn0
            call kkenergy ( rx(:,i,k), k, i,
:              kkk )
              kenew=kenew+kkk
            enddo
          endif
        enddo
        deltke=kenew-keold

      c      calculate the acceptance probability
      call acc_p(deltv,deltke,ps)
      ps=ps*ga

      c      accept/reject
      if (ps.gt.ranf(dummy)) then

      c      restricted path integral
      if(.not.ifb)goto 41
      do i=1,ftn0
        call ifdm(rs,i,ifdms)
        if ( ifdms .lt. epsilon ) goto 1234
      enddo

      41      continue

      ke=ke+deltke
      v=w+deltv
      w=w+deltw
      goto 1235
    endif
  1234      continue

      call equiv(rx,r0)
      acatmab = acatmab - 1.d0
      if (ip.ne.kp) then
        call switch(next(ip),next(kp))
        call switch(next(cp,ip),next(cp,kp))
        acatmas = acatmas - 1.d0
      endif
    1235      continue

      c      build the permutations cycles in text
      call permcyc()

      c      count number of swaps of two particles
      call cycles(cyc,nswap)

      c      balance ke between bridge and displace
      c      call sumupke ( ke )
      c      call sumupot ( potk, overlap, v, w )

      IF (STEP.GT.IEQUI) THEN
        ACM = ACM + 1.0

      C      ** CALCULATE INSTANTANEOUS VALUES **
      IF (POTK .EQ. 'LJ') THEN
        VN = ( V + VLRC )
      ELSE
        VN = V - W
        WN = W
      ENDIF
      CALL CWIN( WIND )
      IF (WIND.GT.3*BETA*IC/2/FTM**2) WIND=3*BETA*IC/2/FTM**2

```

```

C  ** ACCUMULATE AVERAGES **
      ACV  = ACV  + VN
      ACVSQ = ACVSQ + VN*VN
      ACW  = ACW  + WN
      ACWSQ = ACWSQ + WN*WN
      ACKE  = ACKE  + KE
      ACKESQ = ACKESQ + KE*KE
      ACWIND = ACWIND + WIND
    ENDIF

    IF (IFBRIDGE) GOTO 97

C  *****
C  ** ENDS BRIDGE&SWAP MOVE
C  *****

77  CONTINUE

C  *****
C  ** DISPLACEMENT MOVE
C  *****

C  ** PREVIOUS IP **
      DO I=1,NP
        J=NEXT(I)
        PREV(J)=I
      ENDDO

      NIP = NEXT(IP)
      PIP = PREV(IP)

      DO 90 C1 = 1, FTNO
        DO IDIM = 1, DIM
          RXIOLD(IDIM) = RX(IDIM,C1,IP)
        ENDDO

C  ** CALCULATE THE ENERGY OF I IN THE OLD CONFIGURATION **
        CALL PENERGY ( POTK, RXIOLD, IP, C1,
          :           VOLD, WOLD )

        CALL KENERGY ( RXIOLD, IP, C1,
          :           KEOLD )

C  ** INSTANTANEOUS VALUE OF THE ACTION **
        ACTOLD = KEOLD + VOLD

C  ** MOVE I AND PICKUP THE CENTRAL IMAGE **
        IDIM = 1
        RXINEW(IDIM) = RXIOLD(IDIM) +
          :           ( 2.0 * RANF ( DUMMY ) - 1.0 ) * DRMAX(1)
        DO IDIM = 2, DIM
          RXINEW(IDIM) = RXIOLD(IDIM) +
          :           ( 2.0 * RANF ( DUMMY ) - 1.0 ) * DRMAX(IDIM)
          RXINEW(IDIM) = RXINEW(IDIM) -
          :           ANINT ( RXINEW(IDIM)/SIGM(IDIM-1) ) * SIGM(IDIM-1)
        ENDDO

C  ** CONSIDER ONLY THE THIN WHITE DWRF ATMOSPHERE **
        IF (RXINEW(1).GT.RADIUS.OR.RXINEW(1).LT.RADIUS-DELTA) THEN
          GOTO 1238
        ENDIF

C  ** CALCULATE THE ENERGY OF I IN THE NEW CONFIGURATION **
        CALL PENERGY ( POTK, RXINEW, IP, C1,
          :           VNEW, WNEW )

        CALL KENERGY ( RXINEW, IP, C1,
          :           KENEW )

C  ** INSTANTANEOUS VALUE OF THE ACTION **
        ACTNEW = KENEW + VNEW

        DELTV = VNEW - VOLD
        DELTW = WNEW - WOLD
        DELTKE = KENEW - KEOLD

C  ** CHECK FOR ACCEPTANCE **
        DELTACT = ACTNEW - ACTOLD
        DELTACTB = LS*DELTACT

        IF ( EXP ( - DELTACTB ) .GT. RANF ( DUMMY ) ) THEN
          V      = V + DELTV
          W      = W + DELTW
          KE      = KE + DELTKE
          ACATMA = ACATMA + 1.0

          DO IDIM = 1, DIM
            RX(IDIM,C1,IP) = RXINEW(IDIM)
C  IMAGINARY TIME PERIODIC BOUNDARY CONDITIONS
            IF (C1.EQ.1) THEN
              RX(IDIM,FTNO+1,PIP)=RX(IDIM,C1,IP)
            ENDIF
            IF (C1.EQ.FTNO) THEN
              RX(IDIM,0,NIP)=RX(IDIM,C1,IP)
            ENDIF
          ENDDO

C  RESTRICTED PATH INTEGRAL
          IF(.NOT.IFB)GOTO 42
          DO I=1,FTNO
            CALL IFDMN(RS,I,IFDMS)
            IF ( IFDMS .LT. EPSILON ) GOTO 1236
          ENDDO
          CONTINUE
          GOTO 1237
        CONTINUE
        V      = V - DELTV
        W      = W - DELTW
        KE      = KE - DELTKE
        ACATMA = ACATMA - 1.0

        DO IDIM = 1, DIM
          RX(IDIM,C1,IP) = RXIOLD(IDIM)
          IF (C1.EQ.1) THEN
            RX(IDIM,FTNO+1,PIP)=RX(IDIM,C1,IP)
          ENDIF
          IF (C1.EQ.FTNO) THEN
            RX(IDIM,0,NIP)=RX(IDIM,C1,IP)
          ENDIF
        ENDDO
        CONTINUE
        GOTO 1237
      ENDIF
      CONTINUE
    ENDIF
  IF (STEP.GT.IEQUI) THEN
    ACM = ACM + 1.0
  ENDIF

C  ** CALCULATE INSTANTANEOUS VALUES **
C  HISTOGRAM FOR g(r)
  IF ( MOD ( C1, ICALCG ) .EQ. 0 ) THEN
    CALL GOFRC( C1, MAXBIN, HIST, STEP )
  ENDIF

  IF (POTK .EQ. 'LJ') THEN
    VN = ( V + VLRC )
  ELSE
    VN = V - W
    WN = W
  ENDIF
  CALL CWIND( WIND )
  IF (WIND.GT.3*BETA*IC*2/FTM**2) WIND=3*BETA*IC*2/FTM**2

C  ** ACCUMULATE AVERAGES **
      ACV  = ACV  + VN
      ACVSQ = ACVSQ + VN*VN
      ACW  = ACW  + WN
      ACWSQ = ACWSQ + WN*WN
      ACKE  = ACKE  + KE
      ACKESQ = ACKESQ + KE*KE
      ACWIND = ACWIND + WIND
    ENDIF

C  *****
C  ** END DISPLACEMENT MOVE
C  *****

90  CONTINUE

C  *****
C  ** ENDS LOOP OVER TIME SLICES
C  *****

97  CONTINUE

C  ** WRITE OUT THE INSTANTANEOUS VALUES ON FORT.9 **
      SFI = WIND *FTM**2/BETA/IC/2
      SF  = ACWIND*FTM**2/BETA/IC/2/ACM

      CALL FLUSH(9)
      IF ( MOD ( STEP, IPRINT ) .EQ. 0 ) THEN
        WRITE(9,*) STEP,KE/FTNO,VN/FTNO,WN/FTNO,SFI,NSWAP
      ENDIF

C  ** CREATE POSITION DISTRIBUTION **
C  CALL DISTR(30_8,STEP*FTNO*NP/30)

C  ** PERFORM PERIODIC OPERATIONS **
      IF ( MOD ( STEP, IRATIO ) .EQ. 0 ) THEN

C  ** ADJUST MAXIMUM DISPLACEMENT **
        RATIO = ACATMA / DBLE ( FTNO * IRATIO )
        RATIOB = ACATMAB / DBLE ( IRATIO )
        RATIOS = ACATMAS / DBLE ( IRATIO )

        IF ( RATIO .GT. 0.5 ) THEN

C  DRMAX(1) = DRMAX(1) * 1.05
        ELSE

C  DRMAX(1) = DRMAX(1) * 0.95
        ENDIF

        ACATMA = 0.0
        ACATMAB = 0.0
        ACATMAS = 0.0
      ENDIF

      IF ( MOD ( STEP, IPRINT ) .EQ. 0 ) THEN

C  ** WRITE OUT RUNTIME INFORMATION **
        WRITE(*,*) '          step          acm dr/sig  ard

```

```

:   arb      arg'
      WRITE(*,'(2(2XI13),(2Xf6.4),4(2Xf6.6))') STEP, INT(ACM),
:           ALPHA(1), RATIO, RATIOB, RATIOS
:   WRITE(*,*) '           ke           v           w
:   sf'
      WRITE(*,'(4(2XE13.7))') KE/FTNO, VN/FTNO, WN/FTNO,
:   WRITE(*,*) '           <ke>           <v>           <w>
:   <sf>'
      WRITE(*,'(4(2XE13.7))') ACKE/ACM/FTNO, ACV/ACM/FTNO,
:   ACW/ACM/FTNO, SF
      WRITE(*,*) '           nswap'
      WRITE(*,'(I3)') NSWAP
      ENDIF

      IF ( MOD ( STEP, ISAVE ) .EQ. 0 ) THEN
C   ** WRITE OUT THE CONFIGURATION AT INTERVALS **

      CALL WRITCN ( CNFILE )
      REWIND(16)
      CALL FLUSH(16)
      WRITE(16,*)(HIST(I),I=1,MAXBIN),
&      ACV,ACVSQ,ACW,ACWSQ,ACKE,ACKESQ,ACWIND,ACM,STEP

C   ** WRITE OUT THE g(r) **

      IF ( STEP .GT. IEQUI ) THEN
        TSTEP = DBLE((STEP - IEQUI)*INT(FTNO/ICALCG))
        REWIND(15)
        DO BIN = 1, MAXBIN
          RLOWER = DBLE(BIN - .5)*DELR
          RUPPER = RLOWER + DELR
          NIDEAL = 4*PI*DENS*
&          (RUPPER**DIM-RLOWER**DIM)/3
          GR = DBLE(HIST(BIN))/TSTEP/DBLE(NP)/NIDEAL
          CALL FLUSH(15)
          WRITE (15,*) DBLE(BIN-1)*DELR, GR
        ENDDO
      ENDIF

      ENDIF

100  CONTINUE

C   *****
C   ** ENDS THE LOOP OVER CYCLES **
C   *****

      WRITE(*,'(///' END OF MARKOV CHAIN          '///)')

C   ** CHECKS FINAL VALUE OF THE POTENTIAL ENERGY IS CONSISTENT **

      CALL SUMUP (POTK, OVRLAP, KEEND, VEND, WEND)

      WRITE(*,'('' FINAL V           '' ,E10.4)') VEND/FTNO
      WRITE(*,'('' FINAL W           '' ,E10.4)') WEND/FTNO
      WRITE(*,'('' FINAL KE          '' ,E10.4)') KEEND/FTNO

      IF ( ABS((VEND - V)/V) .GT. 1.0d-01 ) THEN

        WRITE(*,'('' PROBLEM WITH V ENERGY !!!)')
        WRITE(*,'('' VEND   = '' , E20.6)') VEND
        WRITE(*,'('' V     = '' , E20.6)') V

      ENDIF

      IF ( ABS((KEEND - KE)/KE) .GT. 1.0d-01 ) THEN

        WRITE(*,'('' PROBLEM WITH KE ENERGY !!!)')
        WRITE(*,'('' KEEND = '' , E20.6)') KEEND
        WRITE(*,'('' KE   = '' , E20.6)') KE

      ENDIF

C   ** WRITE OUT THE FINAL CONFIGURATION FROM THE RUN **

      CALL WRITCN ( CNFILE )

C   ** CALCULATE AND WRITE OUT RUNNING AVERAGES **

      AVV  = ACV / ACM
      ACVSQ = ( ACVSQ / ACM ) - AVV ** 2
      AVKE  = ACKE / ACM
      ACKESQ = ( ACKESQ / ACM ) - AVKE ** 2

C   ** CALCULATE FLUCTUATIONS **

      IF ( ACVSQ .GT. 0.0 ) FLV = SQRT ( ACVSQ/ACM )/FTNO
      IF ( ACKESQ .GT. 0.0 ) FLKE = SQRT ( ACKESQ/ACM )/FTNO

      WRITE(*,'('' AVERAGES '' / )')
      WRITE(*,'('' <V/N>           = '' ,E12.6)') AVV/FTNO
      WRITE(*,'('' <KE/N>          = '' ,E12.6)') AVKE/FTNO

      WRITE(*,'('' FLUCTUATIONS '' / )')

      WRITE(*,'('' FLUCTUATION IN <V/N> = '' ,E12.6)') FLV
      WRITE(*,'('' FLUCTUATION IN <KE/N> = '' ,E12.6)') FLKE
      WRITE(*,'('' END OF SIMULATION '' / )')

C   *****
C   ** THE END **
C   *****

      CLOSE(UNIT = 9 )
      CLOSE(UNIT = 15)
      CLOSE(UNIT = 16)

      STOP
      END

      subroutine permcyc()
      implicit none
      ! given a particle i, lext(i) returns the cycle it belongs to
      INCLUDE 'mc-wd.par'

      integer*8 i,j,k,ii(mnp),jj,kk,ll

      kk=1
      do i=1,np
        ll=1
        ii(ll)=i
        ! 1 jj=next(ii(ll))
        do j=1,ll
          if(jj.eq.ii(j))goto 2
        enddo
        ll=ll+1
        ii(ll)=jj
        goto 1
        ! 2 do k=1,ll
        do k=1,ll
          lext(ii(k))=kk
        enddo
        kk=kk+1
      enddo

      return
      end

      subroutine cycles(cyc,ns)
      implicit none
      ! cyc(i) returns the number of particles in cycle of type i
      ! ns returns the total number of 2 particles swaps
      INCLUDE 'mc-wd.par'
      integer*8 i,cyc(mnp),ns

      ns=0
      cyc=0
      do i=1,np
        cyc(lex(i))=cyc(lex(i))+1
      enddo
      do i=1,np
        if (cyc(i).ne.0) ns=ns+cyc(i)-1
      enddo

      return
      end

      subroutine switch(i,j)
      ! switch i and j
      implicit none
      integer*8 i,j,k
      k=i
      i=j
      j=k
      return
      end

      subroutine acc_p(dv,dke,p)
      implicit none
      ! complete acceptance probability
      INCLUDE 'mc-wd.par'

      real*8 p,dke,dv

      p=exp(-ls*dv) ! contribution from the pair potential
      p=p*exp(-ls*dke) ! contribution from the kinetic energy

      return
      end

      subroutine updaten(j,rxp,ip,nip)
      implicit none
      ! updates a portion of the current path x using the proposed path xp
      INCLUDE 'mc-wd.par'

      integer*8 ip,nip,kp,nkp,j,l,k,idim
      integer*8 mcm
      real*8 rxp(mdim,0:n)

      mcm = j+mbm-floor((j+mbm-.1)/ftn0)*ftn0

      l=0
      if(j+mbm.le.ftn0)then
        do k=j+1,mcm-1
          l=l+1
          do idim=1,dim
            rpn(idim,k,ip)=rxp(idim,l)
          enddo
        enddo
      else
        do k=j+1,ftn0
          l=l+1
          do idim=1,dim
            rpn(idim,k,ip)=rxp(idim,l)
          enddo
        enddo
        do k=1,mcm-1
          l=l+1
          do idim=1,dim
            rpn(idim,k,nip)=rxp(idim,l)
          enddo
        enddo
      enddo
      do idim=1,dim

```

```

      rpn(idim,ftn0+1,ip)=rpn(idim,1,nip)
      rpn(idim,0,nip)=rpn(idim,ftn0,ip)
    enddo

    return
  end

  subroutine swapn(j,ip,nip,kp,nkp)
    implicit none
    ! swap the final part of the paths
    INCLUDE 'mc-wd.par'

    integer*8 ip,nip,kp,nkp,j,k,idim
    integer*8 mcm
    real*8 rr

    mcm = j+mbm-floor((j+mbm-.1)/ftn0)*ftn0

    if(j+mbm.le.ftn0)then
      do k=mcm,ftn0
        do idim=1,dim
          rr=rpn(idim,k,ip)
          rpn(idim,k,ip)=rpn(idim,k,kp)
          rpn(idim,k,kp)=rr
        enddo
      enddo
      do idim=1,dim
        rpn(idim,ftn0+1,ip)=rpn(idim,1,nkp)
        rpn(idim,ftn0+1,kp)=rpn(idim,1,nip)
        rpn(idim,0,nip)=rpn(idim,ftn0,kp)
        rpn(idim,0,nkp)=rpn(idim,ftn0,ip)
      enddo
    endif
    return
  end

  subroutine bridge(x0,x1,stdr,stda,xnew,out)
    implicit none
    ! sample n gaussians with std from xnew(0)=x0 to xnew(ftn0)=x1
    INCLUDE 'mc-wd.par'

    integer*8 l1,l2,l3,j,idim,out
    real*8 stdr,stda,d,s,xi
    real*8 x0(mdim),x1(mdim),xnew(mdim,0:n)

    l3=mbm

    idim=1
    xnew(idim,0)=x0(idim)
    xnew(idim,l3)=x0(idim)+(x1(idim)-x0(idim))
    do idim=2,dim
      xnew(idim,0)=x0(idim)
      xnew(idim,l3)=x0(idim)+(x1(idim)-x0(idim))-
        : anint((x1(idim)-x0(idim))/sigm(idim-1))*sigm(idim-1)
    enddo

    do j=1,mbm-1
      l1=j-1
      l2=j
      s=stdr*(dble(l3-l2)/dble(l3-l1))*0.5d0
      idim=1
      d=xnew(idim,l3)-xnew(idim,l1)
      xnew(idim,j)=xnew(idim,l1)+d/dble(l3-l1)*xi(s)
      if(xnew(idim,j).lt.radius-delta.or.
        : xnew(idim,j).gt.radius)out=3
      s=stda*(dble(l3-l2)/dble(l3-l1))*0.5d0
      do idim=2,dim
        d=xnew(idim,l3)-xnew(idim,l1)
        d=d-anint(d/sigm(idim-1))*sigm(idim-1)
        xnew(idim,j)=xnew(idim,l1)+d/dble(l3-l1)*xi(s)
        xnew(idim,j)=xnew(idim,j)-
        : anint(xnew(idim,j)/sigm(idim-1))*sigm(idim-1)
      enddo
    enddo

    return
  end

  function xi(std)
    ! sample a gaussian with standard deviation std (box-muller method)
    implicit none
    real*8 xi,std,pi,ranf
    data pi/3.14159265358979323846264338328d0/
    xi=cos(pi*ranf(0.d0))*std*sqrt(-2.d0*log(tiny(pi)+ranf(0.d0)))
    return
  end

  SUBROUTINE DISTR(NN,NORM)
    IMPLICIT NONE
    C WRITES ON FORT.10 THE X-POSITION DISTRIBUTION
    INCLUDE 'mc-wd.par'

    REAL*8 DS,DIST(0:1000)
    INTEGER*8 NN,NORM,I,J,K
    SAVE DIST

    DS=SIGMA/NN

    DO I=0,NN
      DO J=1,FTNO
        DO K=1,NP
          IF(-SIGMA/2+(I-.5)*DS.LT.RX(1,J,K).AND.
            : RX(1,J,K).LT.-SIGMA/2+(I+.5)*DS) THEN
            DIST(I)=DIST(I)+1.DO
          ENDIF
        ENDDO
      ENDDO
    ENDDO

    WRITE(10,*) -SIGMA/2+I*DS,DIST(I)/NORM
  ENDDO

  CLOSE(UNIT=10)

  RETURN
END

FUNCTION FACT ( N )
  IMPLICIT NONE
  C FACTORIAL FUNCTION
  INTEGER*8 FACT,N,P,I
  P=1
  DO I=1,N
    P=P*I
  ENDDO
  FACT=P
END

SUBROUTINE SUMUP (POTK, OVLAP, KE, V, W)
  IMPLICIT NONE
  C *****
  C ** CALCULATES THE TOTAL ENERGY **
  C ** USAGE: **
  C ** THE SUBROUTINE RETURNS THE TOTAL ENERGY AT THE **
  C ** BEGINNING AND END OF THE RUN. **
  C *****
  INCLUDE 'mc-wd.par'

  REAL*8 V, KE, VV, KK
  LOGICAL OVLAP
  CHARACTER POTK*(*)

  REAL*8 RXII, RXIJ, RYIJ, RZIJ
  REAL*8 VIJ, WIJ, RIJSQ, W, WW, SQR
  REAL*8 RJ(MDIM), RJO(MDIM), OMC1, OMC2

  INTEGER*8 TAU, I, J, IDIM

  C POTENTIAL ACTION

  VV = 0.0
  WW = 0.0

  C ** LOOP OVER ALL THE PAIRS IN THE LIQUID **

  DO TAU = 1, FTNO
    DO I = 1, NP
      SQR = SQRT(1.-2.*MASS/RX(1,TAU,I))
      VV = VV - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
      WW = WW - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
    ENDDO

    DO 100 I = 1, NP - 1
      DO 99 J = I + 1, NP
        RJ(2)=RX(2,TAU,J)
        RJ(3)=RX(3,TAU,J)
        RJO(2)=RX(2,TAU,J)
        RJO(3)=RX(3,TAU,J)
        IF (RJ(2).GT.0) THEN
          RJ(2)=RJ(2)-SIGM(1)
        ELSE
          RJ(2)=RJ(2)+SIGM(1)
        ENDIF
        IF (RJ(3).GT.0) THEN
          RJ(3)=RJ(3)-SIGM(2)
        ELSE
          RJ(3)=RJ(3)+SIGM(2)
        ENDIF
        OMC1 = RX(2,TAU,I)*RX(2,TAU,J)+RX(3,TAU,I)*RX(3,TAU,J)
        OMC2 = RJ(2)*RX(2,TAU,I)+RJ(3)*RX(3,TAU,I)
        IF (OMC1.GT.OMC2) THEN
          RX(2,TAU,J)=RJ(2)
          RX(3,TAU,J)=RJ(3)
        ENDIF

        CALL POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
        VV = VV + VIJ
        WW = WW + WIJ

        RX(2,TAU,J)=RJO(2)
        RX(3,TAU,J)=RJO(3)
      ENDDO
    ENDDO
    V = VV
    W = WW

  C KINETIC ACTION

  KK = 0.0

  DO TAU = 1, FTNO
    DO I = 1, NP
      RXII = RX(1,TAU,I)-RX(1,TAU+1,I)
      KK=KK+FTM*RXII**2./(1.-2.*MASS/RX(1,TAU,I))
      & / (2.DO*LS**2.)

      RXII = RX(2,TAU,I)-RX(2,TAU+1,I)
      RXII = RXII - ANINT ( RXII/SIGM(1) ) * SIGM(1)
      KK=KK+FTM*(RX(1,TAU,I)+RXII)**2./ (2.DO*LS**2.)

      RXII = RX(3,TAU,I)-RX(3,TAU+1,I)
      RXII = RXII - ANINT ( RXII/SIGM(2) ) * SIGM(2)
      KK=KK+FTM*(RX(1,TAU,I)+COS(RX(2,TAU,I))*RXII)**2.
    ENDDO
  ENDDO

```

```

&      / (2.D0*LS**2.)
      ENDDO
      ENDDO
      KE = KK

      RETURN
      END

      SUBROUTINE SUMUPOT (POTK, OVRLAP, V, W)
      IMPLICIT NONE
      *****
      C ** CALCULATES THE TOTAL POTENTIAL ENERGY **
      C ** **
      C ** USAGE: **
      C ** **
      C ** THE SUBROUTINE RETURNS THE TOTAL ENERGY AT THE **
      C ** BEGINNING AND END OF THE RUN. **
      C *****
      INCLUDE 'mc-wd.par'

      REAL*8      V, VV, KK
      LOGICAL      OVRLAP
      CHARACTER    POTK*(*)

      REAL*8      RXII, RXIJ, RYIJ, RZIJ
      REAL*8      VIJ, WIJ, RIJSQ, W, WW, SQR
      REAL*8      RJ(MDIM), RJO(MDIM), OMCG1, OMCG2

      INTEGER*8    TAU, I, J, IDIM

      C  POTENTIAL ACTION

      VV = 0.0
      WW = 0.0

      C  ** LOOP OVER ALL THE PAIRS IN THE LIQUID **

      DO TAU = 1, FTWO
      DO I = 1, NP
      SQR = SQRT(1.-2.*MASS/RX(1,TAU,I))
      VV = VV - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
      WW = WW - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
      ENDDO

      DO 100 I = 1, NP - 1
      DO 99 J = I + 1, NP

      RJ(2)=RX(2,TAU,J)
      RJ(3)=RX(3,TAU,J)
      RJO(2)=RX(2,TAU,J)
      RJO(3)=RX(3,TAU,J)
      IF (RJ(2).GT.0) THEN
      RJ(2)=RJ(2)-SIGM(1)
      ELSE
      RJ(2)=RJ(2)+SIGM(1)
      ENDIF
      IF (RJ(3).GT.0) THEN
      RJ(3)=RJ(3)-SIGM(2)
      ELSE
      RJ(3)=RJ(3)+SIGM(2)
      ENDIF
      OMCG1 = RX(2,TAU,I)*RX(2,TAU,J)+RX(3,TAU,I)*RX(3,TAU,J)
      OMCG2 = RJ(2)*RX(2,TAU,I)+RJ(3)*RX(3,TAU,I)
      IF (OMCG1.GT.OMCG2) THEN
      RX(2,TAU,J)=RJ(2)
      RX(3,TAU,J)=RJ(3)
      ENDIF

      CALL POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
      VV = VV + VIJ
      WW = WW + WIJ

      RX(2,TAU,J)=RJO(2)
      RX(3,TAU,J)=RJO(3)

      99      CONTINUE
      100     CONTINUE
      ENDDO
      V = VV
      W = WW

      RETURN
      END

      SUBROUTINE PPENERGY ( POTK, RPI, I, RPJ, J,
      :      TAU, V, W )
      IMPLICIT NONE
      *****
      C ** RETURNS THE POTENTIAL ENERGY OF ATOMS I AND J WITH **
      C ** ALL OTHER ATOMS PLUS THE ONE BETWEEN I AND J **
      C ** **
      C ** PRINCIPAL VARIABLES: **
      C ** **
      C ** INTEGER I          THE ATOM OF INTEREST **
      C ** INTEGER NP         THE NUMBER OF ATOMS **
      C ** INTEGER TAU        THE TIMESTEP **
      C ** REAL*8  RX, RY, RZ  THE ATOM POSITIONS **
      C ** REAL*8  RXI,RYI,RZI THE COORDINATES OF ATOM I **
      C ** REAL*8  V           THE POTENTIAL ENERGY OF ATOM I **
      C ** REAL*8  W           THE VIRIAL OF ATOM I **
      C ** **
      C ** USAGE: **
      C ** **
      C ** THIS SUBROUTINE IS USED TO CALCULATE THE CHANGE OF ENERGY **
      C ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND **
      C ** AFTER THE RANDOM DISPLACEMENT OF I. **
      C *****
      INCLUDE 'mc-wd.par'

      REAL*8      RXI(MDIM), V, W
      INTEGER*8    I, J, TAU, IDIM
      CHARACTER    POTK*(*)

      REAL*8      RXIJ, RYIJ, RZIJ, RIJSQ, VIJ, WIJ, SQR
      REAL*8      RXIO(MDIM), RJ(MDIM), RJO(MDIM), OMCG1, OMCG2

      V = 0.0
      W = 0.0

      C  MAPPING
      RXI=RPJ
      RXJ=RPJ

      V = 0.0
      W = 0.0

      DO IDIM = 1, DIM
      RXIO(IDIM) = RX(IDIM,TAU,I)
      RX(IDIM,TAU,I) = RXI(IDIM)
      RXJO(IDIM) = RX(IDIM,TAU,J)
      RX(IDIM,TAU,J) = RXJ(IDIM)
      ENDDO

      SQR = SQRT(1.-2.*MASS/RX(1,TAU,I))
      SQRJ = SQRT(1.-2.*MASS/RX(1,TAU,J))
      V = V + 8*PI*SIG*DENS*
      & (RX(1,TAU,I)-(RADIUS-DELTA))**2*
      & (RX(1,TAU,I)+2*(RADIUS-DELTA))/3/RX(1,TAU,I)
      IF (I.NE.J) THEN
      V = V + 8*PI*SIG*DENS*
      & (RX(1,TAU,J)-(RADIUS-DELTA))**2*
      & (RX(1,TAU,J)+2*(RADIUS-DELTA))/3/RX(1,TAU,J)
      ENDIF
      V = V - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS
      W = W - LOG(ABS(RX(1,TAU,I)**2*COS(RX(2,TAU,I))/SQR))/LS

      C  ** LOOP OVER ALL MOLECULES EXCEPT I AND J **

      DO 100 K = 1, NP
      IF ( K .NE. I .AND. K .NE. J ) THEN

      CALL POT (TAU, I, K, RIJSQ, VIJ, WIJ, POTK)
      V = V + VIJ
      W = W + WIJ

      IF ( I .NE. J ) THEN

      CALL POT (TAU, J, K, RIJSQ, VIJ, WIJ, POTK)
      V = V + VIJ
      W = W + WIJ

      ENDIF

      ENDIF

      100 CONTINUE

      C  RETURN

      IF ( I .NE. J ) THEN

      CALL POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
      V = V + VIJ
      W = W + WIJ

      C  *****

      V = V - LOG(ABS(RX(1,TAU,J)**2*COS(RX(2,TAU,J))/SQRJ))/LS
      W = W - LOG(ABS(RX(1,TAU,J)**2*COS(RX(2,TAU,J))/SQRJ))/LS
      ENDIF

      DO IDIM = 1, DIM
      RX(IDIM,TAU,I) = RXIO(IDIM)
      RX(IDIM,TAU,J) = RXJO(IDIM)
      ENDDO

      RETURN
      END

      SUBROUTINE PENERGY ( POTK, RXI, I, TAU,
      :      V, W )
      IMPLICIT NONE
      *****
      C ** RETURNS THE POTENTIAL ENERGY OF ATOM I WITH ALL OTHER ATOMS. **
      C ** **
      C ** PRINCIPAL VARIABLES: **
      C ** **
      C ** INTEGER I          THE ATOM OF INTEREST **
      C ** INTEGER NP         THE NUMBER OF ATOMS **
      C ** INTEGER TAU        THE TIMESTEP **
      C ** REAL*8  RX, RY, RZ  THE ATOM POSITIONS **
      C ** REAL*8  RXI,RYI,RZI THE COORDINATES OF ATOM I **
      C ** REAL*8  V           THE POTENTIAL ENERGY OF ATOM I **
      C ** REAL*8  W           THE VIRIAL OF ATOM I **
      C ** **
      C ** USAGE: **
      C ** **
      C ** THIS SUBROUTINE IS USED TO CALCULATE THE CHANGE OF ENERGY **
      C ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND **
      C ** AFTER THE RANDOM DISPLACEMENT OF I. **
      C *****
      INCLUDE 'mc-wd.par'

      REAL*8      RXI(MDIM), V, W
      INTEGER*8    I, J, TAU, IDIM
      CHARACTER    POTK*(*)

      REAL*8      RXIJ, RYIJ, RZIJ, RIJSQ, VIJ, WIJ, SQR
      REAL*8      RXIO(MDIM), RJ(MDIM), RJO(MDIM), OMCG1, OMCG2

      V = 0.0
      W = 0.0

```

```

DO IDIM = 1, DIM
  RXIO(IDIM) = RX(IDIM,TAU,I)
  RX(IDIM,TAU,I) = RXI(IDIM)
ENDDO

SQR = SQRT(1.-2.*MASS/RX(1,TAU,I))
V = V - LOG(ABS(RX(1,TAU,I)**2+COS(RX(2,TAU,I))/(SQR)))/LS
W = W - LOG(ABS(RX(1,TAU,I)**2+COS(RX(2,TAU,I))/(SQR)))/LS

C ** LOOP OVER ALL MOLECULES EXCEPT I **

DO J = 1, NP
  IF ( I .NE. J ) THEN

    RJ(2)=RX(2,TAU,J)
    RJ(3)=RX(3,TAU,J)
    RJO(2)=RX(2,TAU,J)
    RJO(3)=RX(3,TAU,J)
    IF (RJ(2).GT.0) THEN
      RJ(2)=RJ(2)-SIGM(1)
    ELSE
      RJ(2)=RJ(2)+SIGM(1)
    ENDIF
    IF (RJ(3).GT.0) THEN
      RJ(3)=RJ(3)-SIGM(2)
    ELSE
      RJ(3)=RJ(3)+SIGM(2)
    ENDIF
    OMCG1 = RX(2,TAU,I)*RX(2,TAU,J)+RX(3,TAU,I)*RX(3,TAU,J)
    OMCG2 = RJ(2)*RX(2,TAU,I)+RJ(3)*RX(3,TAU,I)
    IF (OMCG1.GT.OMCG2) THEN
      RX(2,TAU,J)=RJ(2)
      RX(3,TAU,J)=RJ(3)
    ENDIF

    CALL POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
    V = V + VIJ
    W = W + WIJ

    RX(2,TAU,J)=RJO(2)
    RX(3,TAU,J)=RJO(3)
  ENDIF
ENDDO

DO IDIM = 1, DIM
  RX(IDIM,TAU,I) = RXIO(IDIM)
ENDDO

RETURN
END

SUBROUTINE KENERGY ( RXI, I, TAU, KE )
  IMPLICIT NONE
  *****
  ** RETURNS THE KINETIC ENERGY OF ATOM I WITH ALL OTHER ATOMS. **
  **
  ** PRINCIPAL VARIABLES:
  **
  ** INTEGER I          THE ATOM OF INTEREST
  ** INTEGER NP         THE NUMBER OF ATOMS
  ** INTEGER TAU        THE TIMESTEP
  ** REAL*8  RX, RY, RZ THE ATOM POSITIONS
  ** REAL*8  RXI,RYI,RZI THE COORDINATES OF ATOM I
  ** REAL*8  KE         THE KINETIC ENERGY OF ATOM I
  **
  ** USAGE:
  **
  ** THIS SUBROUTINE IS USED TO CALCULATE THE CHANGE OF ENERGY
  ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND
  ** AFTER THE RANDOM DISPLACEMENT OF I.
  *****
  INCLUDE 'mc-wd.par'

  REAL*8  RXI(MDIM), KE
  REAL*8  RXIS, RXII
  INTEGER*8 IP, TAU, IDIM

  KE = 0.d0

  RXIS = RX(1,TAU+1,IP)
  RXII = RXI(1) - RXIS
  KE=KE+FTM*RXII**2./(1.-2.*MASS/RXI(1))/(2.DO*LS**2.)

  RXIS = RX(2,TAU+1,IP)
  RXII = RXI(2) - RXIS
  RXII = RXII - ANINT ( RXII/SIGM(1) ) *SIGM(1)
  KE=KE+0.5*FTM*(RXI(1)*RXII/LS)**2.

  RXIS = RX(3,TAU+1,IP)
  RXII = RXI(3) - RXIS
  RXII = RXII - ANINT ( RXII/SIGM(2) ) *SIGM(2)
  KE=KE+0.5*FTM*(RXI(1)*COS(RXI(2))*RXII/LS)**2.

  RETURN
END

SUBROUTINE CWIND ( WIND )
  IMPLICIT NONE
  *****
  ** RETURNS THE WINDING NUMBER.
  *****
  INCLUDE 'mc-wd.par'

  REAL*8  WIND, WI, SURF
  REAL*8  RX11, RX21, RX31, RX12, RX22, RX32
  INTEGER*8 I, J, IDIM

  WIND = 0.d0
  SURF = 4*PI*RADIUS**2
  DO I = 1, NP
    DO J = 1, FTNO - 1
      RX11 = RX(1,J ,I)
      RX21 = RX(1,J+1,I)
      RX31 = RX(1,J+2,I)
      RX12 = RX(2,J ,I)
      RX22 = RX(2,J+1,I)
      RX32 = RX(2,J+2,I)
      WI = ((RX21-RX11)*(RX32-RX22)-(RX22-RX12)*(RX31-RX21))*
        : RADIUS**2*COS(RX21)
      WIND = WIND + WI
    ENDDO
  ENDDO
  IF (WIND.GT.SURF) WIND = SURF
  WIND = WIND**2

20  CONTINUE
  RETURN
END

REAL*8 FUNCTION RANF ( DUMMY )
  IMPLICIT NONE
  *****
  ** RETURNS A UNIFORM RANDOM VARIATE IN THE RANGE 0 TO 1.
  **
  **
  ** *****
  **
  ** ** WARNING **
  **
  ** *****
  **
  ** GOOD RANDOM NUMBER GENERATORS ARE MACHINE SPECIFIC.
  ** PLEASE USE THE ONE RECOMMENDED FOR YOUR MACHINE.
  **
  ** RAND(FLAG) returns a pseudo-random number from a uniform
  ** distribution between 0 and 1. If FLAG is 0, the next number
  ** in the current sequence is returned; if FLAG is 1, the
  ** generator is restarted by CALL SRAND(0); if FLAG has any
  ** other value, it is used as a new seed with SRAND.
  *****
  INTEGER*8 L, C, M
  PARAMETER ( L = 1029, C = 221591, M = 1048576 )
  INTEGER*8 SEED
  SAVE SEED
  DATA SEED / 0 /
  REAL*8 DUMMY

  SEED = MOD ( SEED * L + C, M )
  RANF = DBLE( SEED ) / M

  RANF = RAND ( )

  RETURN

```

```

END

SUBROUTINE READCN ( CNFILE )
  IMPLICIT NONE
C *****
C ** SUBROUTINE TO READ IN THE CONFIGURATION FROM UNIT 10 **
C *****
  INCLUDE 'mc-wd.par'

  CHARACTER CNFILE*(*), HASH*1, MYFMT*77

  INTEGER*8 CNUNIT, I, J, NNP, NI, IDIM
  PARAMETER ( CNUNIT = 10 )

C *****

  OPEN ( UNIT = CNUNIT, FILE = CNFILE, STATUS = 'OLD' )
  WRITE(MYFMT,'(A,I10,A)') '(I3,3X,',DIM,'(D12.6,3X))'

  READ ( CNUNIT,* ) HASH, FTNO, NNP
  IF ( NNP.NE. NP ) STOP 'N ERROR IN READCN'

  DO 100 I = 1, NNP
    READ ( CNUNIT,* ) HASH, NEXT(I)
    NI = NEXT(I)
    DO 90 J = 1, FTNO+1
      READ ( CNUNIT, MYFMT ) LEXT(I), (RX(IDIM,J,I),IDIM=1,DIM)
      RX(1,J,I) = RX(1,J,I) + RADIUS - DELTA
    90 ENDDO
    READ ( CNUNIT,13 )
    READ ( CNUNIT,13 )
    DO IDIM = 1, DIM
      RX(IDIM,0,NI) = RX(IDIM,FTNO,I)
    ENDDO
  100 ENDDO
  13 FORMAT(A2)

  CLOSE ( UNIT = CNUNIT )

  RETURN
END

SUBROUTINE WRITCN ( CNFILE )
  IMPLICIT NONE
C *****
C ** SUBROUTINE TO WRITE OUT THE CONFIGURATION TO UNIT 10 **
C *****
  INCLUDE 'mc-wd.par'

  CHARACTER CNFILE*(*), MYFMT*77

  INTEGER*8 CNUNIT, I, J, IDIM
  PARAMETER ( CNUNIT = 10 )

C *****

  OPEN ( UNIT = CNUNIT, FILE = CNFILE, STATUS = 'UNKNOWN' )
  WRITE(MYFMT,'(A,I10,A)') '(I3,3X,',DIM,'(D12.6,3X))'

  WRITE(*,*) 'output to file -----'
  :-----
  CALL FLUSH ( CNUNIT )
  WRITE ( CNUNIT,* ) '#',FTNO,NP

  DO I = 1, NP
    WRITE ( CNUNIT,* ) '#', NEXT(I)
    DO J = 1, FTNO+1
      WRITE ( CNUNIT, MYFMT ) LEXT(I), RADIUS-RX(1,J,I),
      : (RX(IDIM,J,I),IDIM=2,DIM)
    ENDDO
    WRITE ( CNUNIT,13 ) ''
    WRITE ( CNUNIT,13 ) ''
  ENDDO
  13 FORMAT(A2)

  CLOSE ( UNIT = CNUNIT )

  RETURN
END

SUBROUTINE INITCNSPH ( CNFILE )
  IMPLICIT NONE
C *****
C ** SUBROUTINE TO INITIALIZE THE CONFIGURATION TO UNIT 10 **
C *****
  INCLUDE 'mc-wd.par'

  REAL*8 RANF
  CHARACTER CNFILE*(*), MYFMT*77

  INTEGER*8 CNUNIT, I, J, IDIM
  REAL*8 RXI(DIM)
  PARAMETER ( CNUNIT = 10 )

C *****

  OPEN ( UNIT = CNUNIT, FILE = CNFILE, STATUS = 'UNKNOWN' )
  WRITE(MYFMT,'(A,I10,A)') '(I3,3X,',DIM,'(D12.6,3X))'

  WRITE ( CNUNIT,* ) '#',FTNO,NP

  DO I = 1, NP
    WRITE ( CNUNIT,* ) '#', NEXT(I)
    RXI(1) = 0.
    RXI(2) = SIGM(2)*(RANF(0.d0)-.5)
    DO J = 1, FTNO+1
      WRITE ( CNUNIT, MYFMT ) I, RXI(1), RXI(2)
    ENDDO
  ENDDO
  13 FORMAT(A2)

  CLOSE ( UNIT = CNUNIT )

  RETURN
END

SUBROUTINE INITCNWD ( CNFILE )
  IMPLICIT NONE
C *****
C ** SUBROUTINE TO INITIALIZE THE CONFIGURATION TO UNIT 10 **
C *****
  INCLUDE 'mc-wd.par'

  REAL*8 RANF
  CHARACTER CNFILE*(*), MYFMT*77

  INTEGER*8 CNUNIT, I, J, IDIM
  REAL*8 RXI(DIM)
  PARAMETER ( CNUNIT = 10 )

C *****

  OPEN ( UNIT = CNUNIT, FILE = CNFILE, STATUS = 'UNKNOWN' )
  WRITE(MYFMT,'(A,I10,A)') '(I3,3X,',DIM,'(D12.6,3X))'

  WRITE ( CNUNIT,* ) '#',FTNO,NP

  DO I = 1, NP
    WRITE ( CNUNIT,* ) '#', NEXT(I)
    RXI(1) = DELTA*RANF(0.d0)
    RXI(2) = SIGM(1)*(RANF(0.d0)-.5)
    RXI(3) = SIGM(2)*(RANF(0.d0)-.5)
    DO J = 1, FTNO+1
      WRITE ( CNUNIT, MYFMT ) I, RXI(1), RXI(2), RXI(3)
    ENDDO
    WRITE ( CNUNIT,13 ) ''
    WRITE ( CNUNIT,13 ) ''
  ENDDO
  13 FORMAT(A2)

  CLOSE ( UNIT = CNUNIT )

  RETURN
END

SUBROUTINE POT (TAU, I, J, RIJSQ, VIJ, WIJ, POTK)
  IMPLICIT NONE
C *****
C ** SUBROUTINE FOR THE PAIR POTENTIAL **
C *****
  INCLUDE 'mc-wd.par'

  CHARACTER POTK*(*)
  INTEGER*8 TAU, I, J
  REAL*8 RIJ, RIJSQ, RMIN, RMSQ, RCSQ, SR2, SR6, VIJ, WIJ
  REAL*8 COU3, QHS3, F
  REAL*8 R1I, R1J, R2I, R2J, R3I, R3J, VIJR, CG, SG, SG2
  PARAMETER ( COU3 = 4.76015472795910701328763470057d0 )
  PARAMETER ( QHS3 = 24.d0 )

C PI = ACOS(-1.d0)

  VIJ = 0.d0
  WIJ = 0.d0
  IF (DIM.GT. 3) THEN
    WRITE(*,*) "DIM > 3 !!!"
    STOP
  ENDIF
  IF (POTK.EQ. 'FREE') RETURN

  IF (POTK.EQ. 'LJ') THEN
    CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
    C W. L. McMillan, Phys. Rev. A 138, 442 (1965) C
    CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
    C RMIN = 2.d0**(1./6)
    C RMIN = EPSA
    C RMIN = TINY(PI)
    RMIN = 0.d0
    RIJSQ = RIJSQ / (SIG * SIG)
    RMSQ = RMIN*RMIN
    RCSQ = RCUT*RCUT
    IF (RIJSQ.LT. RMSQ) THEN
      RIJSQ = RMSQ
      SR2 = 1.d0 / RIJSQ
      SR6 = SR2 * SR2 * SR2
      VIJ = SR6 * ( SR6 - 1.d0 )
      WIJ = SR6 * ( SR6 - 5.d-1 )
      VIJ = 4.d0 * EPS * VIJ
      WIJ = 48.d0 * EPS * WIJ / 3.d0
    ELSEIF (RIJSQ.GT. RMSQ .AND. RIJSQ.LT. RCSQ) THEN
      SR2 = 1.d0 / RIJSQ
      SR6 = SR2 * SR2 * SR2
      VIJ = SR6 * ( SR6 - 1.d0 )
      WIJ = SR6 * ( SR6 - 5.d-1 )
      VIJ = 4.d0 * EPS * VIJ
      WIJ = 48.d0 * EPS * WIJ / 3.d0
    ENDIF
  ELSEIF (POTK.EQ. 'BUMP') THEN
    RIJ = SQRT( RIJSQ )
    IF (RIJ.LE. SIG) THEN
      VIJ = EPSR
    ENDIF
  ELSEIF (POTK.EQ. 'PSW') THEN
    RIJ = SQRT( RIJSQ )
    IF (RIJ.LE. SIG) THEN
      VIJ = EPSR
    ENDIF
  ENDIF

```

```

ELSEIF (RIJ .LE. SIG + RCUT .AND. RIJ .GT. SIG) THEN
  VIJ = -EPSA
ENDIF
ELSEIF (POTK .EQ. 'COULOMB') THEN
  F = NP/(NP-1)
  RIJ = SQRT( RIJSQ )
  IF (RIJ .GT. TINY(PI)) THEN
    IF (DIM .EQ. 2) THEN
      VIJ = -F*COU3*SIG/SIGMA/2.
      VIJ = VIJ + SIG/RIJ
    ELSEIF (DIM .EQ. 3) THEN
      VIJ = -F*(6.-PI*LOG(4.))-4.*LOG(SIGMA/SIG)/4.
      VIJ = VIJ - LOG(RIJ/SIG)
    ELSEIF (DIM .EQ. 1) THEN
      VIJ = F*SIGMA/SIG/4.
      VIJ = VIJ - RIJ/SIG
    ENDIF
  ELSE
    RIJ = TINY(PI)
    IF (DIM .EQ. 2) THEN
      VIJ = -F*COU3*SIG/SIGMA/2.
      VIJ = VIJ + SIG/RIJ
    ELSEIF (DIM .EQ. 3) THEN
      VIJ = -F*(6.-PI*LOG(4.))-4.*LOG(SIGMA/SIG)/4.
      VIJ = VIJ - LOG(RIJ/SIG)
    ELSEIF (DIM .EQ. 1) THEN
      VIJ = F*SIGMA/SIG/4.
      VIJ = VIJ - RIJ/SIG
    ENDIF
  ENDIF
ELSEIF (POTK .EQ. 'QHS') THEN
  F = NP/(NP-1)
  IF (DIM .EQ. 3) THEN
    IF (RIJSQ .LE. SIG*SIG) THEN
      VIJ = 10.**10.
    ELSE
      VIJ = -F*QHS3*(SIGMA-SIG)/SIGMA**3/2./FTM
      VIJ = VIJ + (2.*RIJSQ+SIG**2.)/(RIJSQ-SIG*SIG)**2./2./FTM
      VIJ = -2.*RIJSQ*(RIJSQ+2.*SIG*SIG)/(RIJSQ-SIG*SIG)**3./FTM
    ENDIF
  ELSE
    WRITE(*,*) 'QHS in dimension different from 3'
    STOP
  ENDIF
ELSEIF (POTK .EQ. 'HARMONIC') THEN
  VIJ = 0.5*RIJSQ/SIG**2.
ELSEIF (POTK .EQ. 'WD') THEN
  R1I=RX(1,TAU,I)-MASS
  R1J=RX(1,TAU,J)-MASS
  R2I=RX(2,TAU,I)
  R2J=RX(2,TAU,J)
  R3I=RX(3,TAU,I)
  R3J=RX(3,TAU,J)
c full expression
  CG = SIN(RX(2,TAU,I))*SIN(RX(2,TAU,J))+
  & COS(RX(2,TAU,I))*COS(RX(2,TAU,J))*
  & COS(RX(3,TAU,I)-RX(3,TAU,J))
c zeroth order
  CG = 1
c first order
  CG = 1 + R2I*R2J + R3I*R3J
c second order
  CG = 1 + R2I*R2J + R3I*R3J - R3J**2/2. +
  - R3I**2*(-0.5 + R3J**2/4.) +
  - R2J**2*(-0.5 - (R3I*R3J)/2. + R3J**2/4. +
  - R3I**2*(0.25 - R3J**2/8.)) +
  - R2I**2*(-0.5 - (R3I*R3J)/2. + R3J**2/4. +
  - R3I**2*(0.25 - R3J**2/8.)) +
  - R2J**2*(0.25 + (R3I*R3J)/4. - R3J**2/8. +
  - R3I**2*(-0.125 + R3J**2/16.))
c full expression
  SG = SQRT(1-CG**2)
c zeroth order
  SG2 = 0
c first order
  SG2 = -2*R3I*R3J + R2I*R2J*(-2 - 2*R3I*R3J)
c second order
  SG2 = -2*R3I*R3J + R3J**2 + R3I**2*(1 - 2*R3J**2) +
  - R2I*R2J*(-2 - 2*R3I*R3J + R3J**2 +
  - R3I**2*(1 - R3J**2/2.)) +
  - R2J**2*(1 + 2*R3I*R3J - R3J**2 +
  - R3I**2*(-1 + 2*R3J**2)) +
  - R2I**2*(1 + 2*R3I*R3J - R3J**2 +
  - R3I**2*(-1 + 2*R3J**2) +
  - R2J**2*(-2 - 2*R3I*R3J + R3J**2 +
  - R3I**2*(1 - 2*R3J**2)))
  SG = SQRT(ABS(SG2))
c full expression
  VIJ = (SIG/RX(1,TAU,I)/RX(1,TAU,J))*
  & (MASS+((RX(1,TAU,I)-MASS)*(RX(1,TAU,J)-MASS)-MASS**2*CG)/
  & SQRT(ABS((RX(1,TAU,I)-MASS)**2+(RX(1,TAU,J)-MASS)**2-
  & 2*(RX(1,TAU,I)-MASS)*(RX(1,TAU,J)-MASS)*CG-MASS**2*SG**2)))
c zeroth order
  VIJR = (R1I-R1J)**2
c first order
  VIJR = (R1I-R1J)**2 - 2*R1I*R1J*R2I*R2J -
  - 2*R1I*R1J*R3I*R3J
c second order
  VIJR = (R1I-R1J)**2 - 2*R1I*R1J*R2I*R2J -
  - 2*R1I*R1J*R3I*R3J + R1I*R1J*R3J**2 +
  - R3I**2*(R1I-R1J - (R1I*R1J*R3J**2)/2.) +
  - R2J**2*(R1I-R1J + R1I*R1J*R3I*R3J -
  - (R1I*R1J*R3J**2)/2. +
  - R3I**2*(-0.5*(R1I-R1J) + (R1I*R1J*R3J**2)/4.)) +
  - R2I**2*(R1I-R1J + R1I*R1J*R3I*R3J -
  - (R1I*R1J*R3J**2)/2. +
  - R3I**2*(-0.5*(R1I-R1J) + (R1I*R1J*R3J**2)/4.)) +
  - R2J**2*(-0.5*(R1I-R1J) - (R1I*R1J*R3I*R3J)/2. +
  - (R1I*R1J*R3J**2)/4. +
  - (R1I*R1J*R3J**2)/4.

```

```

c - R3I**2*((R1I-R1J)/4. - (R1I-R1J*R3J**2)/8.)))
  VIJ = (SIG/R1I/R1J)*(MASS+(R1I-R1J-MASS**2*CG)/
  & SQRT(ABS(VIJR-MASS**2*SG**2)))
ENDIF
RETURN
END
SUBROUTINE GOFR ( TAU, MAXBIN, HIST, STEP )
  IMPLICIT NONE
  *****
  ** CALCULATES THE RADIAL DISTRIBUTION FUNCTION HISTOGRAM. **
  *****
  INCLUDE 'mc-wd.par'
  REAL*8 RIJSQ, RIJ
  INTEGER*8 TAU, IDIM, I, J, BIN, MAXBIN, HIST(MAXBIN)
  INTEGER*8 STEP
  REAL*8 R1I, R1J, R2I, R2J, R3I, R3J, CG
  DO 10 I = 1, NP - 1
    DO 20 J = I + 1, NP
      CALCULATE HISTOGRAM FOR g(r)
      Euclidean Distance
      R1I=RX(1,TAU,I)
      R1J=RX(1,TAU,J)
      R2I=RX(2,TAU,I)
      R2J=RX(2,TAU,J)
      R3I=RX(3,TAU,I)
      R3J=RX(3,TAU,J)
      CG = COS(R2I)*COS(R2J)*COS(R3I)*COS(R3J)+
      & COS(R2I)*COS(R2J)*SIN(R3I)*SIN(R3J)+
      & SIN(R2I)*SIN(R2J)
      c full expression
      RIJSQ = R1I**2+R1J**2-2*R1I*R1J*CG
      c zeroth order
      RIJSQ =(R1I-R1J)**2
      c first order
      RIJSQ = (R1I-R1J)**2 - 2*R1I*R1J*R2I*R2J -
      - 2*R1I*R1J*R3I*R3J
      c second order
      RIJSQ = (R1I-R1J)**2 - 2*R1I*R1J*R2I*R2J -
      - 2*R1I*R1J*R3I*R3J + R1I*R1J*R3J**2 +
      - R3I**2*(R1I-R1J - (R1I*R1J*R3J**2)/2.) +
      - R2J**2*(R1I-R1J + R1I*R1J*R3I*R3J -
      - (R1I*R1J*R3J**2)/2. +
      - R3I**2*(-0.5*(R1I-R1J) + (R1I*R1J*R3J**2)/4.)) +
      - R2I**2*(R1I-R1J + R1I*R1J*R3I*R3J -
      - (R1I*R1J*R3J**2)/2. +
      - R3I**2*(-0.5*(R1I-R1J) + (R1I*R1J*R3J**2)/4.)) +
      - R2J**2*(-0.5*(R1I-R1J) - (R1I*R1J*R3I*R3J)/2. +
      - (R1I*R1J*R3J**2)/4. +
      - R3I**2*((R1I-R1J)/4. - (R1I-R1J*R3J**2)/8.)))
      RIJ = SQRT(ABS(RIJSQ))
      BIN = AINT ( RIJ/DELR ) + 1
      IF (BIN .LE. MAXBIN) THEN
        HIST(BIN) = HIST(BIN) + 2
      ENDIF
    20 CONTINUE
  10 CONTINUE
  RETURN
END
SUBROUTINE MAPPINGD()
  IMPLICIT NONE
  *****
  ** CHANGE COORDINATES (DIRECT) **
  *****
  INCLUDE 'mc-wd.par'
  INTEGER*8 I,J
  DO I=1,NP
    DO J=0,FTNO+1
      RP(1,J,I)=RX(1,J,I)
      RP(2,J,I)=RX(2,J,I)
      RP(3,J,I)=RX(3,J,I)
    ENDDO
  ENDDO
  RETURN
END
SUBROUTINE MAPPINGI()
  IMPLICIT NONE
  *****
  ** CHANGE COORDINATES (INVERSE) **
  *****
  INCLUDE 'mc-wd.par'
  INTEGER*8 I,J
  DO I=1,NP
    DO J=0,FTNO+1
      RX(1,J,I)=RP(1,J,I)
      RX(2,J,I)=RP(2,J,I)
      RX(3,J,I)=RP(3,J,I)
    ENDDO
  ENDDO

```

```

RETURN
END

SUBROUTINE GAUSS(CP,GA)
IMPLICIT NONE
*****
C ** GAUSSIAN TRANSITION PROBABILITY FOR BROWNIAN BRIDGE **
C *****
INCLUDE 'mc-wd.par'

INTEGER*8 I,J,K,CP,MCM
REAL*8 GA,GAR,GAA,VAR,VAA,DD

MCM = CP+MBM-FLOOR((CP+MBM-.1)/FTNO)*FTNO
VAR = 2*LS/FTM
VAA = 2*LS/FTM/RADIUS**2.

GAR = 0.DO
GAA = 0.DO

I=1
DO J=1,NP
  IF(CP+MBM.LE.FTNO)THEN
    DO K=CP+1,MCM
      DD=RPW(I,K,J)-RPW(I,K-1,J)
      GAR=GAR+DD**2
      DD=RP(I,K,J)-RP(I,K-1,J)
      GAR=GAR-DD**2
    ENDDO
  ELSE
    DO K=CP+1,FTNO
      DD=RPW(I,K,J)-RPW(I,K-1,J)
      GAR=GAR+DD**2
      DD=RP(I,K,J)-RP(I,K-1,J)
      GAR=GAR-DD**2
    ENDDO
  DO K=1,MCM
    DD=RPW(I,K,J)-RPW(I,K-1,J)
    GAR=GAR+DD**2
    DD=RP(I,K,J)-RP(I,K-1,J)
    GAR=GAR-DD**2
  ENDDO
  ENDDIF
ENDDO

GAR = EXP(GAR/VAR)

DO I=2,DIM
  DO J=1,NP
    IF(CP+MBM.LE.FTNO)THEN
      DO K=CP+1,MCM
        DD=RPW(I,K,J)-RPW(I,K-1,J)
        DD=DD-ANINT ( DD/SIGM(I-1) )*SIGM(I-1)
        GAA=GAA+DD**2
        DD=RP(I,K,J)-RP(I,K-1,J)
        DD=DD-ANINT ( DD/SIGM(I-1) )*SIGM(I-1)
        GAA=GAA-DD**2
      ENDDO
    ELSE
      DO K=CP+1,FTNO
        DD=RPW(I,K,J)-RPW(I,K-1,J)
        DD=DD-ANINT ( DD/SIGM(I-1) )*SIGM(I-1)
        GAA=GAA+DD**2
        DD=RP(I,K,J)-RP(I,K-1,J)
        DD=DD-ANINT ( DD/SIGM(I-1) )*SIGM(I-1)
        GAA=GAA-DD**2
      ENDDO
    DO K=1,MCM
      DD=RPW(I,K,J)-RPW(I,K-1,J)
      DD=DD-ANINT ( DD/SIGM(I-1) )*SIGM(I-1)
      GAA=GAA+DD**2
      DD=RP(I,K,J)-RP(I,K-1,J)
      DD=DD-ANINT ( DD/SIGM(I-1) )*SIGM(I-1)
      GAA=GAA-DD**2
    ENDDO
  ENDDIF
  ENDDO
  GAA = EXP(GAA/VAA)
  GA = GAA*GAR
END

RETURN
END

SUBROUTINE EQUIV(A,B)
IMPLICIT NONE
*****
C ** A = B WITH A, B TWO PATHS **
C *****
INCLUDE 'mc-wd.par'

INTEGER*8 I,J,K
REAL*8 A(MDIM,0:N,MNP),B(MDIM,0:N,MNP)

DO I=1,DIM
  DO J=0,FTNO+1
    DO K=1,NP
      A(I,J,K)=B(I,J,K)
    ENDDO
  ENDDO
ENDDO

RETURN
END

*****
C determines the sign of the free spinless Fermi density matrix
*****
C *****
C dim = spatial dimensions
C nf = number of fermions
C beta = inverse temperature
C itau0= initial time slice
C itau1= final time slice
C sign = sign of the free fermion density matrix
C *****
subroutine ifdm(itau0,itau1,sign)
implicit none
include 'mc-wd.par'
integer*8 itau0,itau1,i,j
real*8 rrv,sum,rvec,DMGT,epsilon,det,sign
real*8 aaa(np,np),beta,lambda

epsilon=1.d-100
beta=ls*abs(itau1-itau0)
lambda=1./2./ftm

if (itau0.eq.itau1) then
  sign=1.
  return
endif

C set up the free particle density matrix
do i=1,np
  do j=1,np
    rrv=sin(rx(1,itau0,j))*sin(rx(1,itau1,i))+
    & cos(rx(1,itau0,j))*cos(rx(1,itau1,i))*
    & cos(rx(2,itau0,j))-rx(2,itau1,i))
    rvec=radius*acos(rrv)
    sum=rvec**2.
    sum=sum/(4*beta*lambda)
    aaa(i,j)=exp(-sum)
    aaa(i,j)=aaa(i,j)/(4*pi*beta*lambda)**(dim/2.)
  enddo
enddo

det=DMGT(epsilon,np,aaa)

C sign of the determinant
sign=det

return
end

*****
** ifdm.f *****
*****
C determines the sign of the free spinless Fermi density matrix
C *****
C dim = spatial dimensions
C nf = number of fermions
C beta = inverse temperature
C itau0= initial time slice
C itau1= final time slice
C sign = sign of the free fermion density matrix
C *****
subroutine ifdmn(itau0,itau1,sign)
implicit none
include 'mc-wd.par'
integer*8 itau0,itau1,i,j
real*8 rrv,sum,rvec,DMGT,epsilon,det,sign
real*8 aaa(np,np),beta,lambda

epsilon=1.d-100
beta=ls*abs(itau1-itau0)
lambda=1./2./ftm

if (itau0.eq.itau1) then
  sign=1.
  return
endif

C set up the free particle density matrix
do i=1,np
  do j=1,np
    use euclidean distance
    rrv=sin(rpn(2,itau0,j))*sin(rpn(2,itau1,i))+
    & cos(rpn(2,itau0,j))*cos(rpn(2,itau1,i))*
    & cos(rpn(3,itau0,j)-rpn(3,itau1,i))
    rvec=rpn(1,itau0,i)**2.+rpn(1,itau0,j)**2.-
    & 2.*rpn(1,itau0,i)*rpn(1,itau0,j)*rrv
    sum=rvec
    sum=sum/(4*beta*lambda)
    aaa(i,j)=exp(-sum)
    aaa(i,j)=aaa(i,j)/(4*pi*beta*lambda)**(dim/2.)
  enddo
enddo

det=DMGT(epsilon,np,aaa)

C sign of the determinant
sign=det

return
end

!The subroutine TSRGT applies to input real square matrix A(n,n) the upper
!triangularization algorithm of Gauss method with full pivoting and keeps
!trace of successive transformations done in integer vectors KP and LP.
!-----
! Input parameters:
! eps precision (real*8)
! n size of A matrix (integer)
! A pointer to input real square matrix (real*8)
! Output parameters:
! it flag=1 if A matrix ok, =0 if A matrix is singular (integer)
! C pointer to table storing main diagonal elements and supra-

```

```

!      diagonal elements of upper triangular matrix and the multi-
!      plying coefficients used during triangularization process
!      KP      table storing informations concerning the column exchanges
!              during process (integer)
!      LP      table storing informations concerning the line exchanges
!              during process (integer)
!-----
!The table C is first initialized to A matrix, then receives at each step k
!of the triangularization process, usefull elements of A matrix at step k for
!k=1,2,...n.
!The variables po(real*8), lo and ko(integer) store respectively pivot at step k,
!its line number and its column number.
!-----

```

```

Subroutine TSRGT(eps, n, A, it, C, Kp, Lp)
  real*8 eps
  integer*8 n,it
  real*8 A(n,n), C(n,n)
  integer*8 Kp(n),Lp(n)
  real*8 po,t0
  C=A
  it=1
  k=1
  do while (it==1.and.k<n)
    po=C(k,k)
    lo=k
    ko=k
    do i=k, n
      do j=k, n
        if (dabs(C(i,j))>dabs(po)) then
          po=C(i,j)
          lo=i
          ko=j
        end if
      end do
    end do
    Lp(k)=lo
    Kp(k)=ko
    if (dabs(po)<eps) then
      it=0
    else
      if (lo.ne.k) then
        do j=k, n
          t0=C(k,j)
          C(k,j)=C(lo,j)
          C(lo,j)=t0
        end do
      end if
      if (ko.ne.k) then
        do i=1, n
          t0=C(i,k)
          C(i,k)=C(i,ko)
          C(i,ko)=t0
        end do
      end if
      do i=k+1, n
        C(i,k)=C(i,k)/po
      end do
      do j=k+1, n
        C(i,j)=C(i,j)-C(i,k)*C(k,j)
      end do
    end do
    k=k+1
  end if
  end do
  if (it==1.and.dabs(C(n,n))<eps) it=0
  return
End
!TSRGT

```

```

!The function DMGT returns the determinant of a real square matrix
!A(n,n) by Gauss method with full pivoting.
!-----

```

```

! Input parameters:
! eps      precision (real*8)
! n        size of A matrix (integer)
! A        pointer to input real square matrix
! Output parameters:
! None
!-----

```

```

!The procedure TSRGT is used to reduce A matrix to an upper triangular matrix.
!Output variables are it(integer), C(n,n), Kp(n) and Lp(n).
!If it=0, matrix A is singular, if it=1, matrix A is regular. Table C contains
!at location i,j (j>=i) the corresponding element of the upper triangular matrix.
!Tables Lp and Kp contain informations relative to exchanges of line or column
!that occurred during the process. For instance, the element number k of Lp is
!an integer <> k if an exchange of line has been made at step k (k=1,2,...n).
!The number of exchanges of lines and columns is stored in integer L. the
!determinant of A matrix is stored in d0 (real*8).
!-----

```

```

  real*8 Function DMGT(eps, n, A)
  integer*8 n, it, Kp(n), Lp(n)
  real*8 eps, A(n,n), C(n,n)
  real*8 d0

  call TSRGT(eps,n,A,it,C,Kp,Lp) !call triangularization subroutine
  if (it==0) then
    d0=0.d0
  else
    d0=1.d0
    do k=1, n
      do i=k, n
        d0=d0*C(k,i)
      end do
      l=0
      do k=1, n-1
        if (Lp(k).ne.k) l=l+1
        if (Kp(k).ne.k) l=l+1
      end do
      if (MOD(l,2).ne.0) d0=-d0 !l is odd
    end if
    DMGT=d0
  return
End

```

```

*****
*** mc-wd.par *****
*****

```

```

REAL*8      PI
REAL*8      DELR
INTEGER*8    MDIM, N, MNP

```

```

PARAMETER ( PI = 3.1415926535897932384626433832840 )

```

```

PARAMETER ( MDIM = 10 ) ! maximum number of dimensions
PARAMETER ( MNP = 1000 ) ! maximum number of particles
PARAMETER ( N = 3000 ) ! maximum number of time slices

```

```

INTEGER*8    DIM
REAL*8      RX(MDIM,0:N,MNP)
REAL*8      RPN(MDIM,0:N,MNP),RP(MDIM,0:N,MNP)
REAL*8      FTM, LS, SIGMA, SIGM(MDIM)
REAL*8      MASS, RADIUS, DELTA, ANG, DENS
REAL*8      SIG, EPSA, EPSR, EPS, RCUT
INTEGER*8    FTNO, NP
INTEGER*8    NEXT(MNP), LEXT(MNP), MEXT(MNP), MEM

! path
COMMON / BLOCK1 / RX, RP, RPN, DELR, DIM
! pair-potential parameters
COMMON / BLOCK2 / SIG, EPSR, EPSA, EPS, RCUT
! mass, timestep, box edge, # timeslices, # particles
COMMON / BLOCK3 / FTM, LS, SIGMA, SIGM, FTNO, NP
! permutations
COMMON / BLOCK4 / NEXT, LEXT, MEXT, MEM
! white dwarf radius, mass, delta, density
COMMON / BLOCK5 / RADIUS, MASS, DELTA, ANG, DENS

```

```

*****
*** data-gr-rs.in *****
*****

```

```

0 number of spatial dimensions      (wd)
3
1 seed of the random sequence      RAND
4
2 if bose                          (T/F)
T
3 number of particles                (<= MNP)
30
4 the potential                     SUBROUTINE POT
WD
5 # time slices = 1/temperature/timestep (< N)
30
6 mass                             (hbar = kb = 1)
4.2d-23
7 # of cycles                       (nstep)
5000000000000000000
8 # of steps between output lines    (iprint)
100
9 # of steps between configuration saves (isave)
100
10 # of steps for equilibration       (iequi)
100
11 # of steps for acceptance ratios    (iratio)
100
12 # of steps for rdf calculation      (icalcg < #5)
10
13 configuration file name
conf.xyz
14 0 init zero, 1 init, 2 rest conf, 3 rest full
1
15 radius of the wd
4.3d40
16 mass of the wd
9.1d37
17 wd atmosphere thickness
4.3d23
18 wd angular extent of the patch
1.d-17
19 temperature                       THERMODYNAMICS
7.d-27
20 maximum displacement/box edge
1.d-1
1.d-1
1.d-1
21 maximum # of bridge timeslices     (> 1; <= #5)
20
22 potential cutoff distance (LJ)     SUBROUTINE POT
2.d0
23 sig (LJ 2.566)                     SUBROUTINE POT
0.00729735
24 epsr (LJ INIT)                     SUBROUTINE POT
1.d10
25 epsa                               SUBROUTINE POT
1.d0
26 eps (LJ 10.22)                     SUBROUTINE POT
1.d1
27 if only displace                    (T/F)
F
28 if only bridge                      (T/F)
F

```

AUTHOR DECLARATIONS

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Conflicts of interest

None declared.

Funding

None declared.
